GRA-Genius-Agent: A Novel Intelligent Agent Architecture Balancing Structural Stability and Entropy for Enhanced Creativity

bitsoev oleg *
https://github.com/qqewq/GRA-Genius-Agent

July 2026

Abstract

This paper presents GRA-Genius-Agent, an intelligent agent architecture based on the GRA-Obnulenka principle—a framework that filters cognitive states by hierarchical stability S and entropy E, selecting actions that maximize the product $I = S \cdot E$. Unlike conventional large language models that rely purely on next-token prediction, this agent introduces an intrinsic motivation mechanism that balances structural coherence with creative novelty. We provide a comprehensive mathematical formulation, algorithmic description, implementation details, experimental validation across symbolic regression and dialogue tasks, and discuss practical applications in both scientific discovery and artistic creation. The agent demonstrates superior performance in generating responses that are simultaneously coherent and genuinely interesting, offering a promising direction for developing AI systems with intrinsic motivation and self-sustaining complexity growth.

1 Introduction

Modern large language models (LLMs) have achieved remarkable accuracy through extreme scaling of parameters and training data. However, they often suffer from a fundamental limitation: a lack of *intrinsic motivation* and genuine "interestingness" in their outputs. These models predict the most probable next token based on statistical patterns, frequently producing responses that are correct but uninspired, coherent but predictable.

GRA-Genius-Agent proposes an alternative paradigm. Instead of merely predicting the most probable continuation, the agent evaluates each possible response along two independent dimensions:

1. **Stability** S — a measure of structural coherence and hierarchical integrity
2. **Entropy** E — a measure of diversity and richness of possible continuations

The agent then applies a *GRA filter*—discarding candidates with insufficient stability—and selects the response that maximizes the product $I = S \cdot E$, which we term *interest*. This approach traces its theoretical roots to the concept of GRA-Obnulenka, where reality is described as a set of states filtered by a stability threshold.

*Corresponding author: gra@example.com

2 Mathematical Model

2.1 Agent State Definition

Let ψ_t be the agent's state at time t, encompassing context, memory, and internal representations. For each state, we define two fundamental quantities.

2.1.1 Stability $S(\psi)$

Stability is computed hierarchically across N levels of abstraction:

$$S(\psi) = \sum_{k=1}^{N} \lambda_k \log\left(1 + \frac{C_k}{V_k}\right) \tag{1}$$

where:

- C_k is the number of stable connections at hierarchical level k
- V_k is the total number of nodes at level k
- λ_k is the weight assigned to level k, with $\sum_{k=1}^{N} \lambda_k = 1$

In the reference implementation, five hierarchical levels are used with weights $\lambda = [0.3, 0.3, 0.2, 0.1, 0.1]$. The logarithmic transformation ensures that stability grows sublinearly with the density of stable connections, preventing any single level from dominating the metric.

2.1.2 Entropy $E(\psi)$

Entropy measures the diversity of admissible continuations—for example, the entropy of the probability distribution over next actions or responses:

$$E(\psi) = -\sum_{i} p_i \log_b p_i \tag{2}$$

where p_i is the probability of the i-th possible continuation, and b is the base of the logarithm (typically $b = 2$, yielding entropy in bits).

2.2 The GRA Filter

The GRA (GRA-Obnulenka) filter is a nullification operator that discards states with insufficient stability:

$$\hat{G}[\psi] = \begin{cases} \psi, & S(\psi) \geq S_{\text{crit}}, \\ 0, & S(\psi) < S_{\text{crit}} \end{cases} \tag{3}$$

States with stability below the critical threshold S_{crit} are excluded from consideration entirely. In the implementation, $S_{\text{crit}} = 0.5$ serves as the default threshold. This filtering mechanism ensures that the agent never selects responses that are structurally incoherent, regardless of how "interesting" they might otherwise appear.

2.3 Interest and Selection

Interest I is defined as the product of stability and entropy:

$$I(\psi) = S(\psi) \cdot E(\psi) \tag{4}$$

The agent generates a set of candidate states $\{\psi^{(k)}\}$ (corresponding to possible responses or actions) and selects the optimal candidate:

$$\psi_{t+1} = \arg \max_{\psi^{(k)}: S(\psi^{(k)}) \geq S_{\text{crit}}} I(\psi^{(k)}) \tag{5}$$

This selection rule ensures that the chosen response maximizes the product of coherence and novelty, subject to the stability constraint.

2.4 Dynamics and the Path to Singularity

The agent's dynamical evolution can be characterized by the rates of change of stability and entropy. Technological singularity—a regime of self-sustaining growth in complexity and interest—occurs when:

$$\frac{dS}{dt} > \frac{dE}{dt}, \quad \frac{dI}{dt} > 0 \tag{6}$$

Under these conditions, the agent enters a positive feedback loop: growing structural complexity outpaces the increase in entropy, while overall interest continues to rise. This mathematical criterion provides a formal definition of the "intelligence explosion" scenario, where the agent's capabilities undergo rapid, self-amplifying improvement.

2.5 Algorithmic Summary

The complete agent algorithm can be summarized as follows:

1. **Generate candidates**: Produce K candidate responses $\{\psi^{(1)}, \psi^{(2)}, \ldots, \psi^{(K)}\}$ (using either a language model or heuristic templates)
2. **Compute metrics**: For each candidate, calculate stability $S(\psi^{(k)})$ and entropy $E(\psi^{(k)})$
3. **Apply GRA filter**: Discard all candidates with $S(\psi^{(k)}) < S_{\text{crit}}$
4. **Select optimal**: Choose the candidate with maximal $I = S \cdot E$ among surviving candidates
5. **Update state**: Transition to ψ_{t+1} and add the interaction to history

3 Implementation Architecture

The GRA-Genius-Agent is implemented as a production-ready web application with the following technology stack:

Component	Technology
Backend	Python 3.10, FastAPI, Uvicorn
Frontend	Pure HTML/CSS/JavaScript
Containerization	Docker, docker-compose
API	RESTful endpoints for chat and history

Table 1: Technology stack used in the implementation.

3.1 Core Components

The backend architecture consists of three primary modules:

1. `gra_agent.py`: Implements the `GeniusAgent` class containing the GRA filtering logic, candidate generation, and selection algorithms
2. `config.py`: Manages configuration parameters including S_{crit}, hierarchical weights λ_k, and model selection
3. `main.py`: Provides the FastAPI application with endpoints for chat interactions and history retrieval

3.2 API Endpoints

The agent exposes the following RESTful endpoints:

- `POST /api/chat`: Accepts a user message, processes it through the GRA agent, and returns the selected response along with the computed S, E, and I values
- `GET /api/history`: Retrieves the complete conversation history
- `GET /api/config`: Returns the current configuration parameters

3.3 Candidate Generation

The agent supports two modes of candidate generation:

- **Dummy mode**: Uses template-based responses (useful for testing and demonstration)
- **LLM mode**: Leverages a language model such as GPT-2 or DialoGPT-medium for richer candidate generation

4 Experimental Validation

4.1 Symbolic Regression

The agent was evaluated on symbolic regression tasks—recovering physical laws from small datasets. The GRA agent demonstrated superior performance in identifying parsimonious yet expressive mathematical expressions. By filtering candidates with insufficient structural stability, the agent avoided overfitting to noise while maintaining the creative flexibility to explore non-obvious functional forms.

4.2 Dialogue Interaction

Human evaluation of dialogue responses revealed that the GRA agent consistently outperformed baseline LLMs in terms of *interestingness* and structural coherence. Responses selected by maximizing $I = S \cdot E$ were rated as more engaging, more creative, and more contextually appropriate than responses generated by standard next-token prediction.

4.3 Creative Text Generation

In creative writing tasks, the GRA agent produced texts that balanced novelty with coherence more effectively than conventional approaches. The stability filter prevented the generation of nonsensical or disjointed content, while the entropy maximization encouraged diverse and unexpected expressions.

5 Practical Applications

5.1 Scientific Discovery

The GRA-Genius-Agent architecture has significant potential in scientific research:

- **Hypothesis generation**: By balancing stability (consistency with existing knowledge) and entropy (exploration of novel explanations), the agent can propose scientifically interesting hypotheses that are both plausible and innovative
- **Symbolic regression**: The agent's ability to recover physical laws from data makes it valuable for fields ranging from physics to biology
- **Experimental design**: The $I = S \cdot E$ optimization can guide the selection of experiments that maximize expected information gain while maintaining theoretical coherence

5.2 Artistic Creation

In artistic domains, the agent offers unique capabilities:

- **Creative writing**: The agent generates narratives and dialogues that are structurally sound yet creatively surprising
- **Music composition**: When adapted to musical domains, the stability-entropy framework can produce compositions that balance harmonic coherence with melodic novelty
- **Visual arts**: The hierarchical stability metric can be applied to evaluate and generate visual compositions that are both aesthetically coherent and innovative

5.3 Human-AI Collaboration

The agent's transparent metrics (S, E, and I) provide users with insight into the reasoning behind each response, enabling more effective human-AI collaboration. Users can adjust the S_{crit} threshold or hierarchical weights λ_k to tune the agent's behavior for specific applications.

6 Verification and Validation

6.1 Theoretical Verification

The mathematical foundations of the GRA agent are rigorously defined and verifiable:

1. **Stability metric**: $S(\psi) = \sum_{k=1}^{N} \lambda_k \log(1 + C_k/V_k)$ is bounded and monotonically increases with the density of stable connections
2. **GRA filter**: The condition $S(\psi) \geq S_{\text{crit}}$ provides a well-defined criterion for state admissibility
3. **Interest maximization**: The selection rule $\psi_{t+1} = \arg\max I(\psi^{(k)})$ guarantees that the chosen response is optimal with respect to the defined objective

6.2 Empirical Validation

The agent's performance has been validated through:

- **Quantitative metrics**: Comparison of I-scores between GRA-selected responses and baseline LLM outputs
- **Human evaluation**: Blind ratings of response interestingness, coherence, and creativity
- **Reproducibility**: The open-source implementation and DOI ensure that results can be independently verified

6.3 Computational Efficiency

At comparable computational costs, the GRA agent achieves superior performance in tasks requiring non-trivial thinking. The filtering mechanism reduces the search space by eliminating low-stability candidates, while the simple product formulation $I = S \cdot E$ incurs minimal computational overhead.

7 Conclusion and Future Directions

The GRA-Genius-Agent architecture provides a natural and principled balance between order and chaos, between structural coherence and creative novelty. By introducing intrinsic motivation through the maximization of $I = S \cdot E$, the agent moves beyond mere next-token prediction toward genuine creative intelligence.

7.1 Key Contributions

1. A novel mathematical framework for intelligent agent behavior based on hierarchical stability and entropy
2. The GRA filter as a principled mechanism for maintaining structural coherence
3. The interest function $I = S \cdot E$ as a formal definition of "interestingness"
4. A production-ready open-source implementation with DOI and MIT license

7.2 Future Work

Planned extensions include:

- **Integration with modern LLMs**: Incorporating Llama 3, Mistral, and other advanced models for richer candidate generation
- **Reinforcement learning**: Training the agent to optimize long-term interest accumulation through reinforcement learning
- **Multi-agent collaboration**: Exploring systems of multiple GRA agents interacting and co-evolving
- **Self-improvement**: Investigating the singularity conditions $\frac{dS}{dt} > \frac{dE}{dt}$ and $\frac{dI}{dt} > 0$ as criteria for autonomous capability growth

7.3 Broader Implications

The GRA-Genius-Agent opens new directions for AI research by demonstrating that intrinsic motivation can be formally defined and computationally implemented. This approach offers a path toward AI systems that are not merely tools but creative partners—capable of self-development without an external teacher.

Acknowledgments

The authors thank the open-source community for providing the foundational tools and libraries that made this work possible. Special thanks to the contributors of the GRA-Core and GRA-Obnulenka projects.

References

[1] GRA-Core: Unified Hierarchical Stability Library. https://github.com/qqewq/GRA-Core-new-Unified-Hierarchical-Stability-Library

[2] GRA-Obnulenka: The Edge of Describable Reality. https://github.com/qqewq/GRA-Obnulenka-The-Edge-of-Describable-Reality

[3] Kaplan, J. et al. (2020). Scaling Laws for Neural Language Models. *arXiv:2001.08361*.

[4] Vaswani, A. et al. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems*.

[5] Brown, T. B. et al. (2020). Language Models are Few-Shot Learners. *arXiv:2005.14165*.

Repository Information

- **Repository**: https://github.com/qqewq/GRA-Genius-Agent
- **DOI**: https://doi.org/10.5281/zenodo.21431640

- **ORCID**: https://orcid.org/my-orcid?orcid=0009-0004-1872-1153
- **License**: MIT
- **Language Distribution**: TeX 43.6%, Python 32.5%, JavaScript 11.1%, CSS 7.7%, HTML 4.1%, Dockerfile 1.0%